OMNIDB – UNIVERSAL GAME DATABASE

A PROJECT REPORT
Submitted by

BALAMURUGAN RMM – 241701008
VAISHNAV PERUMAL M – 241701061

In fulfilment for the course

GE23627 – DESIGN THINKING AND INNOVATION

for the degree of

BACHELOR OF ENGINEERING IN
COMPUTER SCIENCE AND DESIGN



RAJALAKSHMI ENGINEERING COLLEGE THANDALAM, CHENNAI
OCTOBER 2026

BONAFIDE CERTIFICATE

Certified that this report titled "**OMNI-DB UNIVERSAL GAMEDATABASE**" is the bonafide work of **BALAMURUGAN RMM** (241701008) and **VAISHNAV PERUMAL M** (241701061), who carried out the work under my supervision. Certified further that, to the best of my knowledge, the work reported herein does not form part of any other thesis or dissertation on the basis of which a degree or award was conferred on an earlier occasion on these or any other candidates.

Certified further that the project contributes to SDG 4 – Quality Education, SDG 9 – Industry, Innovation and Infrastructure, and SDG 8 – Decent Work and Economic Growth, with a primary focus on SDG 4 – Quality Education. The project aims to support accessible, interactive, and personalised interview practice through a virtual reality-based mock interview platform that integrates conversational artificial intelligence and immersive simulation, helping students prepare for professional opportunities and develop career readiness..

SIGNATURE OF HOD

Dr. S. Vinodh Kumar, Professor

Head of the Department

SIGNATURE OF SUPERVISOR

Dr. P. Revathy Professor

ABSTRACT

OmniDB – Universal Game Database is a cross-platform game information and discovery application built around the vision “One game. Every platform. One database.” Game information is distributed across digital storefronts and platform ecosystems. Users may need to consult separate services to identify a title, compare its availability and understand the offers associated with different platforms. OmniDB brings catalogue discovery, platform information, multi-store listings, developer and publisher profiles, deal records and wishlist-based price alerts into one interface.

The visible application supports six platform/store categories: Steam, Epic Games, GOG, PlayStation, Xbox and Nintendo. The home dashboard presents a sample multi-store dataset with aggregate listing and deal indicators. The wishlist screen identifies persistent alert storage in a PostgreSQL user_watchlist table and compares user-defined target prices with current lowest deals. The project team reports that frontend and backend development have been completed. This report describes the functions visible in the supplied screenshots and distinguishes them from broader extensions that require separate implementation evidence.

ACKNOWLEDGEMENT

Initially, we thank the Almighty for being with us through every walk of our lives and showering His blessings throughout the endeavour to complete this report. Our sincere thanks to our Chairman Mr. S. Meganathan, B.E., F.I.E., our Vice Chairman Mr. Abhay Shankar Meganathan, B.E., M.S., and our respected Chairperson Dr. (Mrs.) Thangam Meganathan, Ph.D., for providing us with the requisite infrastructure and facilities at their esteemed institution.

Our sincere thanks to Dr. S. N. Murugesan, M.E., Ph.D., our beloved Principal, for his kind support and the facilities provided to complete our project work successfully. We express our sincere thanks to Dr. S. Vinodh Kumar, Ph.D., Professor and Head of the Department of Computer Science and Design, for his valuable guidance and encouragement throughout the project. We convey our heartfelt gratitude to our internal project guide, Dr. P. Revathy, Professor, Department of Computer Science and Design, for her continuous guidance, constructive suggestions and valuable support during the development of our project. We also thank the faculty members, our families and friends for their encouragement and support.

BALAMURUGAN RMM (241701008)

VAISHNAV PERUMAL M (241701061)

TABLE OF CONTENTS

S.NO.	TITLE	PAGE.NO.
1	INTRODUCTION	
2	LITERATURE SURVEY	
3	SYSTEM DESIGN	
4	SOFTWARE IMPLEMENTATION	
5	PLATFORM INTEGRATION AND DATA MANAGEMENT	
6	METHODOLOGY AND ALGORITHMS	
7	TESTING AND RESULTS	
8	CONCLUSION AND FUTURE SCOPE	
	REFERENCES	
	APPENDICES / IMPLEMENTATION EVIDENCE	

LIST OF FIGURES

FIGURE NO.	TITLE	PAGE NO.
3.1	High-level architecture of OmniDB	42
3.2	Context-level data flow	42
3.3	Use case diagram of OmniDB	43
3.4	Activity diagram for game discovery	44
3.5	Sequence diagram for catalogue search	44
3.6	Conceptual entity relationship model	43
4.1	OmniDB home page and navigation dashboard	20
4.2	Cross-platform game catalogue and filters	21
4.3	Universal search interface	22
4.4	Multi-platform price comparison matrix	23
4.5	Developer and publisher directory	24
4.6	Discount listings and historical-low indicators	25
4.7	Wishlist target-price monitoring	26

LIST OF TABLES

TABLE NO.	TITLE	PAGE NO.
2.1	Comparison of existing service categories	14
3.1	Functional requirements	16
3.2	Non-functional requirements	17
3.3	Logical data entities	18
7.1	Functional test cases	34

LIST OF ABBREVIATIONS

S.NO.	ABBREVIATION	EXPANSION
1	API	Application Programming Interface
2	DB	Database
3	ER	Entity Relationship
4	HTTP	Hypertext Transfer Protocol
5	JSON	JavaScript Object Notation
6	UI	User Interface
7	URL	Uniform Resource Locator
8	UX	User Experience

INTRODUCTION

1.1 General

The video game industry is distributed across several digital storefronts, console ecosystems and publisher platforms. A player searching for a particular game may need to visit multiple services to understand where the title is available, which platforms support it, and how its editions or prices differ. Information is therefore fragmented across separate websites and platform-specific catalogues.

OmniDB – Universal Game Database is a cross-platform game information project intended to bring game-related information together in one organised interface. Its product vision is: “One game. Every platform. One database.” The project provides a unified catalogue through which users can discover games and review information associated with supported platforms. The application is designed to make game discovery and comparison more convenient without requiring users to navigate multiple unrelated catalogues.

1.2 Background

Digital distribution has changed how players discover and purchase games. A single title may be released on PC storefronts and several console platforms, while different editions, release information and availability details may be presented separately. Players who own more than one gaming platform may find it difficult to maintain a consolidated view of their interests and purchases.

A universal database approach treats the game as the central record and associates it with platform-specific information. This makes it possible to organise game details in a consistent way and provide features such as catalogue browsing, search, filtering, wishlists, personal libraries and spending awareness. OmniDB applies this approach through a web-based application.

1.3 Problem Statement

Game information is commonly spread across multiple platform-specific stores and databases. Users may need to search several sources to identify a game's platform availability, compare available information and keep track of titles they own or intend to purchase. Separate services can make it difficult to maintain one consistent personal view of games across platforms.

The project addresses the need for a unified, user-friendly game database that organises game information across supported gaming platforms and provides discovery and personal-organisation features in one application.

1.4 Motivation

The project is motivated by the fragmented nature of gaming information and the effort required to move between different storefronts and platform catalogues. A consolidated database can reduce repetitive searching and make it easier for users to explore games across ecosystems. The project also aims to provide personal tools that help users organise their gaming interests and remain aware of their spending on games.

1.5 Objectives

The major objectives of OmniDB are:

1. To develop a unified catalogue for video games across supported platforms.

2. To provide a consistent way to search and browse game information.

3. To organise platform-related information for individual game records.

4. To support comparison of available game information across platforms where data is available.

5. To provide personal organisation features such as a wishlist and game library.

6. To provide a wallet or spending-tracking feature that helps users record and review game expenditure.

7. To implement a maintainable frontend and backend architecture that can be extended as additional authorised data sources become available.

1.6 Scope and Limitations

The project scope is a cross-platform game information and discovery application. It brings together catalogue organisation, multi-store listings, platform associations, developer and publisher information, price comparison, deal discovery and wishlist-based target-price monitoring. The interface identifies Steam, Epic Games, GOG, PlayStation, Xbox and Nintendo as supported platform/store categories. The dashboard labels its records as a sample multi-store dataset; its displayed totals are a snapshot of the data shown during capture, not a limit on the system's capacity or intended scope. The project team reports completion of both frontend and backend development. The supplied screenshots directly

demonstrate the catalogue, search, comparison, studio directory, deals and wishlist modules.

The precise backend framework, database engine, external data providers, deployment environment, authentication method and live price-update mechanism have not been confirmed in the information available for this report. These implementation particulars must be checked against the final source code and deployment before submission. The project should use authorised APIs, licensed datasets or manually curated data; it should not rely on prohibited scraping. Data completeness and freshness depend on the sources integrated into the deployed system.

1.7 Existing System

Existing approaches include individual digital storefronts, platform-specific game libraries, game-information websites and price-comparison services. Each service generally focuses on its own catalogue, platform or specific information category. Users may therefore need to consult more than one source when exploring a game across multiple ecosystems.

1.8 Limitations of Existing Approaches

The main limitations addressed by OmniDB are information fragmentation, inconsistent presentation across platforms, repeated searching, and the separation of personal game records from discovery information. Some sources may also differ in the coverage and freshness of their data. A consolidated interface can improve organisation, but it must clearly communicate the source and update status of information.

1.9 Proposed System

OmniDB proposes a unified game database that presents a canonical catalogue through one application interface. A user can move from discovery to platform comparison and deal inspection without treating each storefront as a separate catalogue. The interface brings together game identity, platform availability, studio information, pricing and user-defined watchlist targets. The design is intended to scale by adding records and authorised data sources while retaining a consistent identity for each game.

The system is developed as a frontend-and-backend application. The frontend provides the user-facing experience, while backend services manage application data and the operations required by the interface. The final report should identify the exact technologies and data

flow from the implemented source code rather than assume a particular framework or database.

1.10 Advantages and Applications

OmniDB can reduce the effort involved in exploring games across separate platform catalogues, provide a single place to organise game interests, and help users review spending recorded in the application. Potential users include PC and console players, multi-platform gamers, game collectors and users who want a consolidated catalogue for discovery. The usefulness of comparisons depends on the availability, accuracy and freshness of the underlying data.

LITERATURE SURVEY

2.1 Introduction

This chapter reviews the problem domain of digital game catalogues, cross-platform discovery, personal game libraries and price information. The objective is to understand the functions offered by existing categories of services and identify the opportunity for a consolidated platform.

2.2 Digital Game Distribution Platforms

Digital distribution platforms provide users with storefronts, game pages, purchase workflows, libraries and platform-specific community features. These services are important primary sources for information about their own catalogues. However, each platform's catalogue and presentation are generally designed around that provider's ecosystem.

2.3 Game Information Databases

Game information databases organise titles and metadata such as descriptions, genres, release details, supported systems and related media. Such databases support discovery and research. Coverage, field definitions and update frequency may vary between providers, making data normalisation important when information from multiple sources is presented together.

2.4 Price Comparison and Availability Information

Price comparison services help users review offers from different retailers or storefronts. Reliable comparison requires consistent identification of the same game and edition, currency handling, region awareness, and clear timestamps. A price should not be presented as current unless it is supported by a current and authorised source. OmniDB's comparison capability should therefore distinguish verified live data from curated, sample or demonstration data.

2.5 Personal Libraries and Wishlists

Personal libraries and wishlists help users organise games they own, have played or intend to purchase. When these features are separated from broader game discovery, users may need to maintain records in more than one place. A unified system can connect catalogue records with personal organisation functions.

2.6 Spending Awareness in Digital Purchases

A spending tracker can help users record purchase amounts and review expenditure over time. The quality of such a feature depends on the accuracy of user-entered or imported transaction information, currency consistency and clear separation between recorded spending and actual financial-account data. OmniDB's wallet concept is treated as a game-spending record, not as a payment wallet unless payment processing is explicitly implemented.

2.7 Comparative Analysis

The following comparison describes broad categories of existing services. It is a conceptual comparison, not a claim that every product in a category has identical capabilities.

SERVICE CATEGORY	PRIMARY PURPOSE	TYPICAL STRENGTH	LIMITATION RELEVANT TO OMNIDB
Digital storefront	Sell and distribute games	Purchase and platform library functions	Usually centred on its own ecosystem
Game information database	Organise game metadata	Searchable game information	May not combine user purchase records
Price comparison service	Compare offers	Offer discovery across retailers	Coverage, region and freshness vary
Personal game library	Track a user's collection	Personal organisation	May be separate from cross-platform discovery
OmniDB	Unified game discovery and personal organisation	Catalogue, platform information, wishlist/library and spending tracker	Data quality depends on verified sources and implementation

2.8 Research Gap

The project opportunity is to provide one interface that connects cross-platform game discovery with personal game organisation and spending awareness. A useful unified system must also address duplicate game identities, differences between editions and platforms, inconsistent metadata, and the reliability of external data. OmniDB seeks to address the user-facing fragmentation through a central catalogue and associated personal features.

2.9 Summary

The review indicates that storefronts, game databases, comparison services and personal libraries each solve important parts of the gaming information problem. OmniDB combines these concerns within a single application concept while recognising that data quality and authorised source integration are essential to trustworthy results.

SYSTEM DESIGN

3.1 General

System design defines the structure of OmniDB and the interaction between its user interface, backend services and persistent data. The application is treated as a client-server system: users interact with the frontend, which communicates with backend functionality to retrieve or update application information.

3.2 Requirement Analysis

The system must support game discovery and the organisation of game records. It should present game information in a consistent format, associate titles with supported platforms, and provide personal functions such as wishlists, libraries and spending records where implemented. It must also handle incomplete or unavailable data gracefully.

3.3 Functional Requirements

The functional requirements for the project are:

1. Display a game catalogue.
2. Allow users to search for games.
3. Provide filtering or browsing controls supported by the application.
4. Display a details view for a selected game.
5. Present platform-related information for a game.
6. Provide price or offer comparison when corresponding data is available.
7. Allow users to maintain a wishlist.
8. Allow users to organise a personal game library.
9. Allow users to record or review game spending through the wallet tracker.
10. Exchange the required data between frontend and backend components.

ID	REQUIREMENT
FR-01	Display game catalogue
FR-02	Search game records
FR-03	Filter or browse catalogue
FR-04	View selected game details
FR-05	Display platform associations
FR-06	Compare available offers where supported
FR-07	Manage wishlist
FR-08	Manage personal library
FR-09	Record and review game spending
FR-10	Communicate with backend services

3.4 Non-Functional Requirements

Usability: the interface should make catalogue browsing and game discovery understandable. Performance: common interactions should respond within an acceptable time under expected usage. Reliability: invalid or unavailable data should not crash the application. Maintainability: modules should be organised so that features can be changed without rewriting unrelated functionality. Security: user-specific records and backend endpoints should be protected according to the application's authentication and authorisation design. Data integrity: game records and platform associations should be represented consistently.

CATEGORY	REQUIREMENT
Usability	Clear navigation and understandable interactions
Performance	Responsive catalogue and search operations
Reliability	Graceful handling of invalid or unavailable data
Maintainability	Modular frontend and backend components
Security	Protect user-specific records and backend endpoints
Data integrity	Consistent game and platform relationships

3.5 Overall System Architecture

The high-level architecture consists of four logical layers:

• Presentation layer – the web frontend, pages, reusable components and user interactions.

• Application/API layer – backend endpoints and business logic used by the frontend.

• Data layer – persistent storage for catalogue records and user-specific information, according to the implemented database.

• Data-source layer – authorised external APIs or curated data inputs, if connected by the deployed application.

The architecture is described at the logical level because the supplied screenshots do not expose the complete source-code stack or API contract. The visible PostgreSQL references support the use of a relational persistence layer for at least the wishlist alert records. The remaining frontend and backend components are represented by their responsibilities rather than by unverified framework names.

3.6 System Flow

A typical discovery flow begins when a user opens the application and views the catalogue. The user searches or filters the available records, selects a game and reviews its details. The application requests the required information from the backend and displays the returned result. If the user chooses to save the title, the relevant wishlist or library operation is submitted and stored. A spending entry, where supported, follows a similar process through the wallet feature.

3.7 Use Case Design

The primary actor is the user. The user's use cases include browse catalogue, search games, filter games, view game details, compare platform information, manage wishlist, manage personal library and record game spending. Administrative or data-maintenance use cases should be added only if they exist in the implemented application.

3.8 Data Flow Design

At a high level, the frontend sends requests to backend services. The backend validates the request, reads or updates the relevant records in persistent storage, and returns a response for presentation. User-specific operations must be associated with the correct user identity where authentication is implemented. External data retrieval, if present, should be isolated behind a service or integration layer.

3.9 Database Design Considerations

A relational representation could include entities such as Game, Platform, GamePlatform, User, WishlistItem, LibraryItem and WalletTransaction. The exact schema must follow the actual database. A many-to-many relationship is appropriate conceptually when one game can be available on multiple platforms and one platform can contain many games. Wishlist and library records associate a user with a game. Spending records should preserve amount, currency, date and the associated game when those fields are captured by the application.

ENTITY (CONCEPTUAL)	PURPOSE
Game	Stores the central game record
Platform	Stores supported platform information
GamePlatform	Associates a game with one or more platforms
User	Represents an application user, if accounts are implemented
WishlistItem	Associates a user with a saved game
LibraryItem	Associates a user with a library game
WalletTransaction	Stores a recorded game-spending entry

3.10 Module Design

The logical modules are:

1. Game catalogue module – presents the collection of game records.

2. Search and discovery module – supports locating titles and narrowing results.

3. Game details module – presents information about a selected title.

4. Platform information module – associates titles with supported platforms.

5. Comparison module – presents comparable platform or offer information when available.

6. Wishlist module – stores titles selected for later consideration.

7. Personal library module – organises a user's game collection.

8. Wallet and spending module – records and summarises game-related expenditure.

9. Backend services module – exposes application operations and manages data access.

The modules documented in this chapter are separated into screenshot-confirmed functions and broader supporting capabilities. The screenshots directly demonstrate the catalogue, search, comparison, studios, deals and wishlist alert modules. Personal library and wallet functions are retained as possible user-centred extensions unless separately demonstrated by implementation evidence.

SOFTWARE IMPLEMENTATION

4.1 Development Environment

The project team reports that the frontend was initially developed with Google AI Studio and that backend development has also been completed. The running interface uses a dark visual theme, a persistent navigation bar, global search controls, game cards and comparison tables. The screenshots expose PostgreSQL-related interface elements, including a PostgreSQL Console, SQL Inspector and a note identifying the user_watchlist table. Exact framework names and version numbers are not inferred from the visual interface.

4.2 Frontend Implementation

The frontend presents a consistent workspace for browsing and analysing game information. The home page combines a global search field, navigation to Games, Deals, Platforms, Studios and Wishlist, and a status strip summarising catalogue and multi-store indicators. The Games view provides platform, genre, franchise, discount and historical-low filters. Search supports title, alternate title, studio, genre and UGDB-ID terms. The interface uses cards, badges, platform markers and compact tables to communicate information without requiring users to visit each storefront separately.

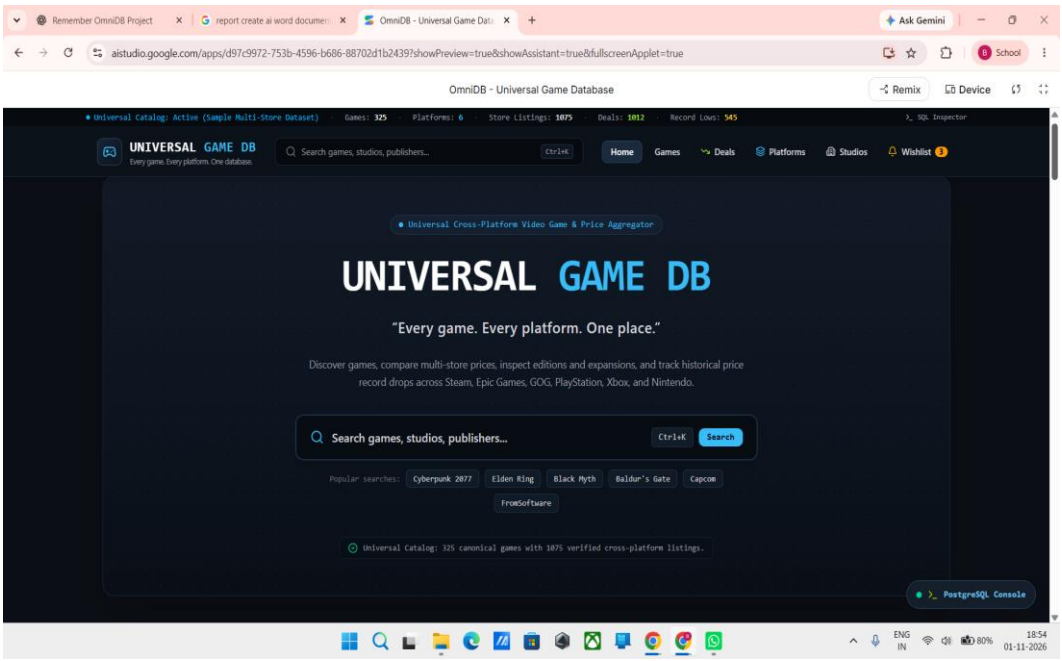


Figure 4.1: OmniDB home page and universal navigation dashboard

4.3 Backend Implementation

The backend is responsible for the data operations that support catalogue retrieval, platform and store associations, deal presentation and user-specific watchlist records. The strongest implementation detail visible in the supplied evidence is the wishlist page statement that persistent alerts are stored in the PostgreSQL user_watchlist table. The PostgreSQL Console and SQL Inspector controls further indicate that the application exposes database-oriented inspection entry points. The exact API routes, server framework and full schema are not visible in the screenshots and are therefore not specified.

4.4 Game Catalogue Management

The catalogue is the central component of OmniDB. The interface presents each title with a UGDB identifier, title, studio, release year, genre, rating, supported platform indicators and price information. These identifiers provide a consistent reference for navigating between catalogue records and related platform or store information. The dashboard also presents aggregate indicators for platform coverage, store listings, deals and historical lows. These values describe the captured sample dataset and are not used to define a maximum catalogue size.

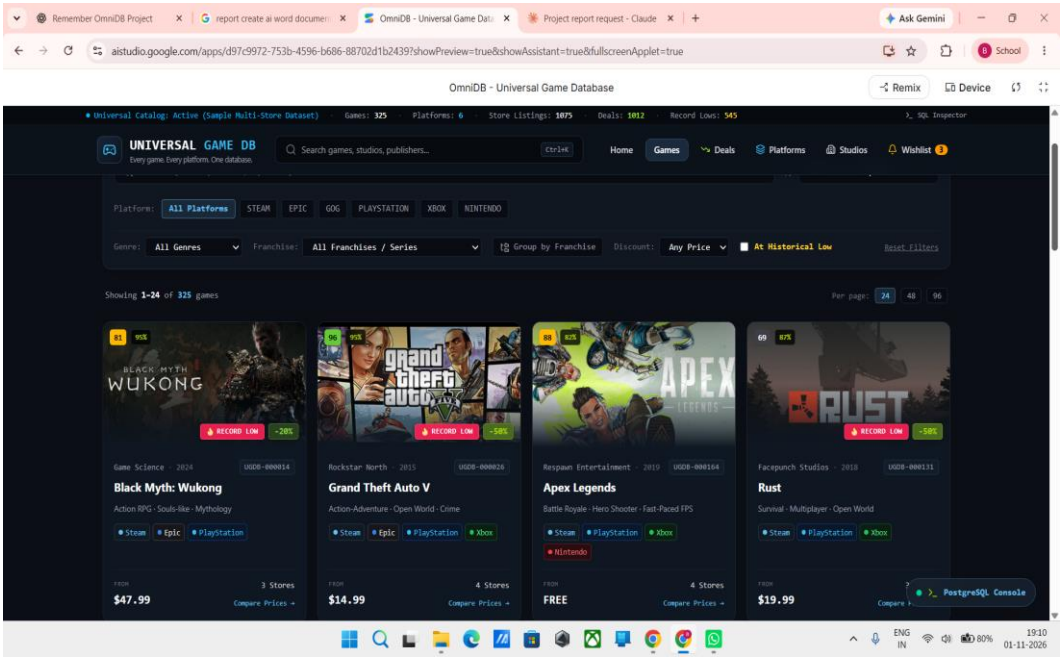


Figure 4.2: Cross-platform game catalogue with discovery filters

4.5 Search and Filtering

Search and filtering provide two complementary discovery paths. The universal search overlay accepts a title, alternate title, studio, genre or UGDB identifier and displays

suggested terms and matching records. The catalogue page adds structured filters for platform, genre, franchise, price/discount and historical-low status. The screenshot also shows pagination controls and options to change the number of records displayed per page. These controls support both direct lookup and exploratory browsing.

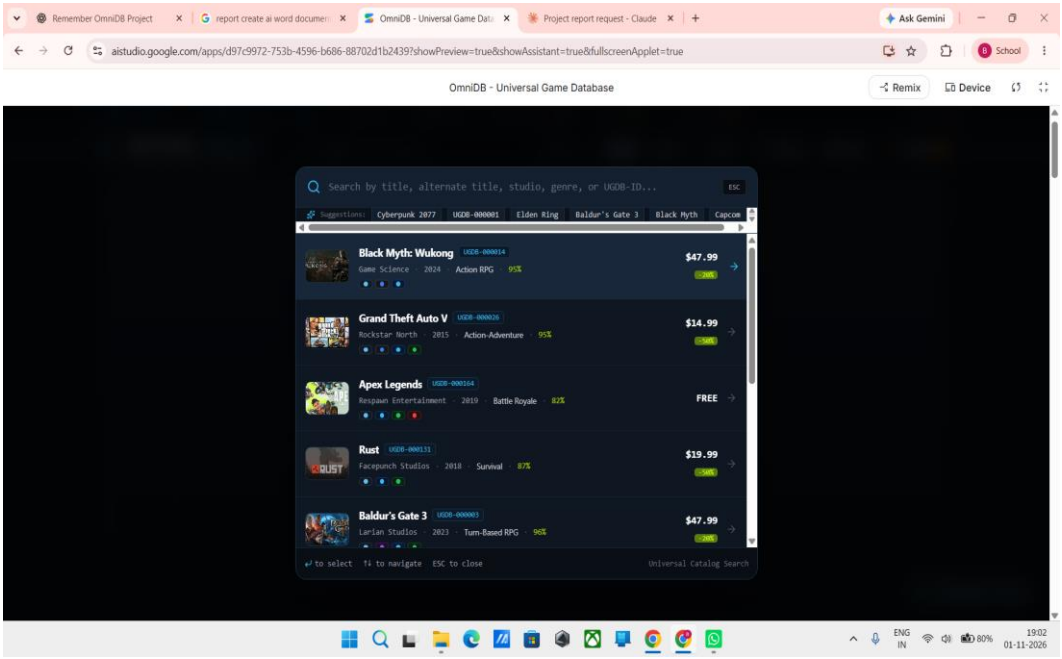


Figure 4.3: Universal search overlay with suggested and matching records

4.6 Game Details and Platform Information

The catalogue and search result cards show the fields available for discovery, including title, UGDB identifier, studio, release year, genre, rating, platform indicators and displayed price. The supplied screenshots do not include a dedicated expanded game-details page. Accordingly, this report describes the visible summary information and the Inspect affordance on the platform comparison screen without claiming details that are not shown.

4.7 Price Comparison

The platform comparison screen places game titles in rows and supported storefront/platform categories in columns. The visible columns are Steam, Epic, GOG, PlayStation, Xbox and Nintendo. Price cells display available amounts, discount badges or an unavailable marker, while the game row also identifies the best visible price. This layout allows a user to compare the same title across multiple ecosystems at a glance. Because prices are time- and region-sensitive, a production-quality comparison should retain currency, region, edition, source and retrieval time for each offer.

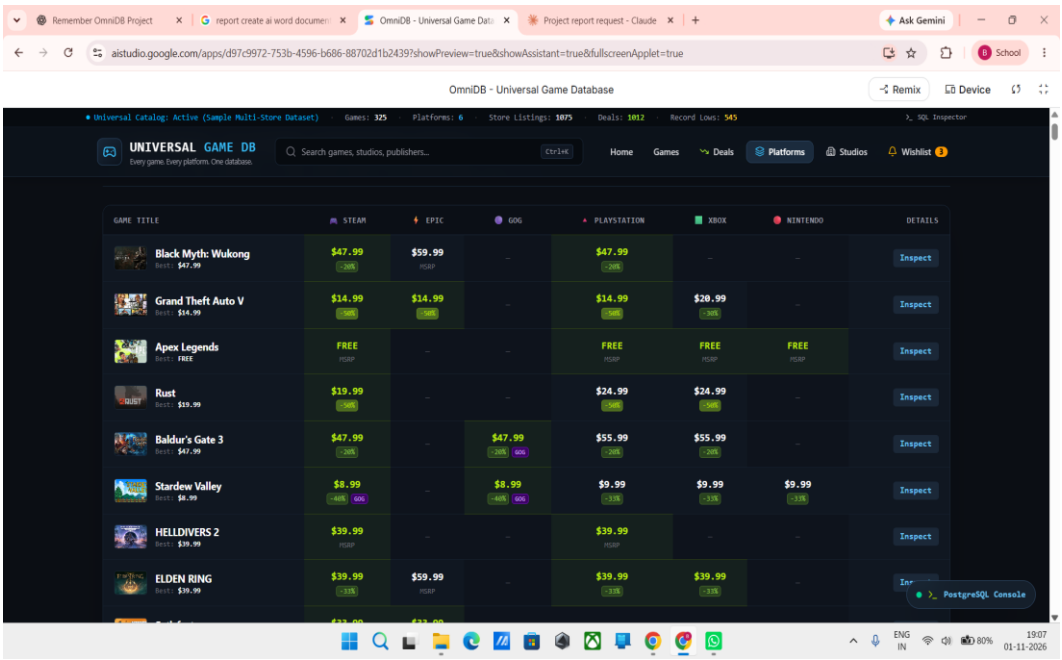


Figure 4.4: Multi-platform price comparison matrix

4.8 Developer and Publisher Directory

OmniDB includes a directory of developer and publisher entities associated with catalogue records. The visible studio cards display a studio identifier, name, country, founding year and the number of games tracked. Each card also provides a catalogue navigation action. The supplied screen includes examples such as Nintendo EPD, Capcom, Square Enix Creative Business Unit, Ubisoft Montreal, TT Games, Game Freak, FromSoftware and BioWare. This directory adds organisational context to the game catalogue and supports discovery by studio.

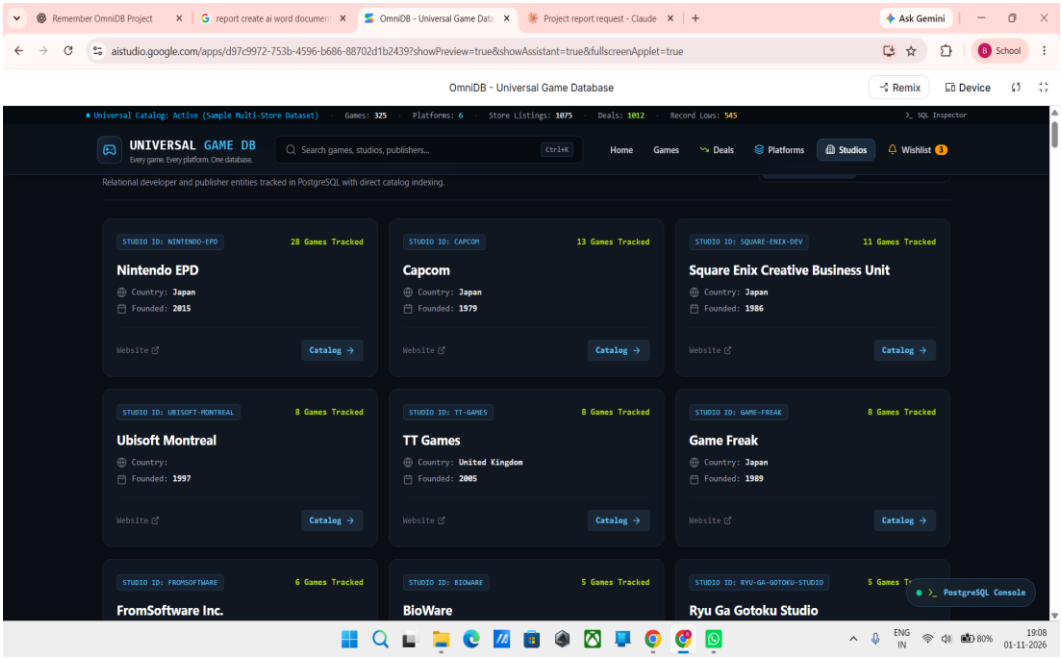


Figure 4.5: Developer and publisher directory

4.9 Deal Discovery and Historical Price Records

The Deals view presents discounted offers as a structured list. The supplied screen shows game titles, platform/store labels, discount percentages, current prices, previous prices, dates associated with the displayed low-price records, DRM/store information and direct Store links. “Lowest Ever” badges distinguish records presented as historical lows. The view supports deal discovery and allows users to review the displayed offer against its previous price. Price records should retain currency, region, edition, source and retrieval time wherever those values are available.

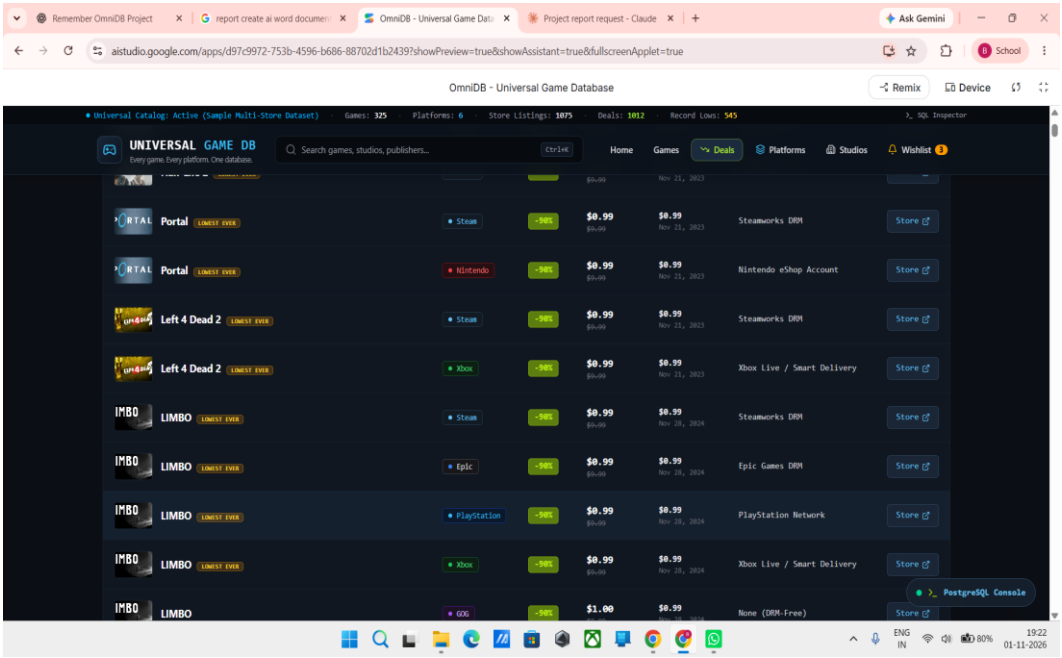


Figure 4.6: Discount listings and historical-low indicators

4.10 Wishlist and Price Alert Management

The wishlist screen is labelled “My Watchlist & Deal Triggers”. Each monitored game displays platform markers, a user target price and the current lowest deal. The screen includes edit and delete controls and a View Deals action. Two visible entries are marked “Target Met”, and the page states that persistent alerts are stored in the PostgreSQL user_watchlist table. This is direct evidence of a user-oriented price-monitoring function rather than a simple list of saved titles.

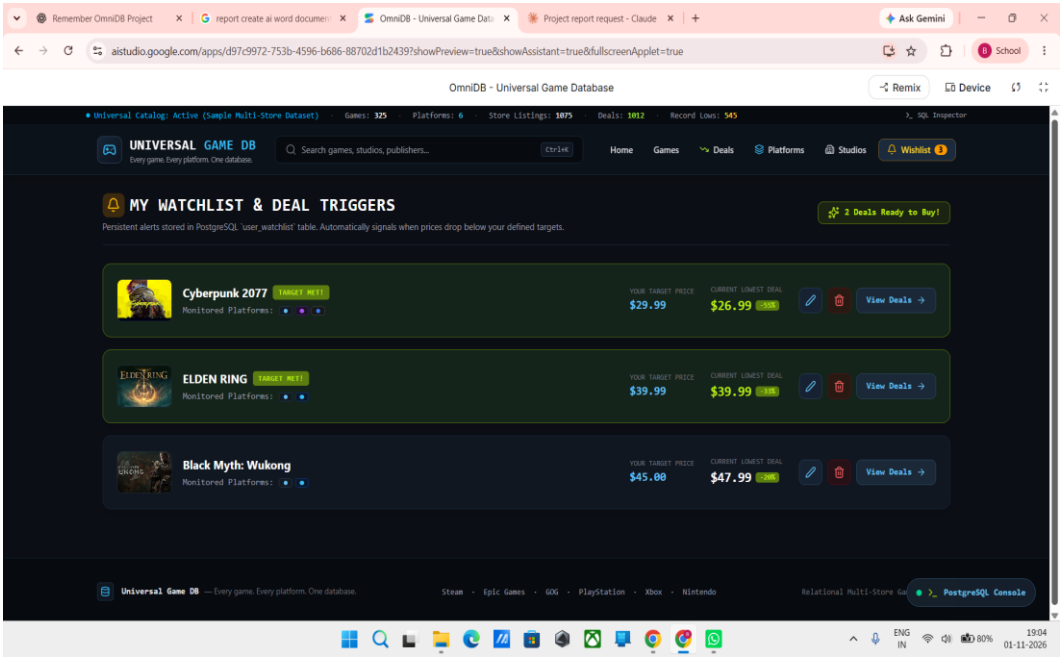


Figure 4.7: Wishlist with target-price monitoring and deal triggers

4.11 Extended Personal Management Features

A wallet or spending tracker has been discussed as part of the wider project direction to help users review game-related expenditure. However, no wallet screen or transaction record was included in the supplied evidence set. It is therefore treated as an extended user-management capability rather than a screenshot-verified function in the present implementation results. If included in the deployed application, it should be documented as a spending record and summary, not as a payment wallet or bank balance unless those capabilities are genuinely implemented.

4.12 User Interface and Accessibility

The interface should maintain consistent navigation, readable text, clear labels and understandable empty or error states. Game images should have suitable alternative text where supported. Controls should be usable across the screen sizes targeted by the application. The final report should include screenshots from the running application rather than illustrative images from unrelated websites.

4.13 Backend and Data Integration

The backend should separate application logic from data access and external integrations. If external providers are used, their names, permitted use, API endpoints and update policy should be documented. If data is locally curated, the report should explain how records are added and maintained. The project should avoid unauthorised scraping and should preserve provenance for externally sourced information.

PLATFORM INTEGRATION AND DATA MANAGEMENT

5.1 Introduction

A cross-platform database must represent a game independently of the platform on which it is played, while preserving platform-specific attributes. This chapter explains the data-management approach required to organise game records and platform associations in OmniDB.

5.2 Cross-Platform Game Representation

A game should have a canonical catalogue identity. Platform availability is represented as a related record rather than creating unrelated copies of the same title for every system. This structure supports a single game details view while allowing platform-specific information to be displayed separately.

5.3 Platform Catalogue Organisation

The application visibly organises information across six platform/store categories: Steam, Epic, GOG, PlayStation, Xbox and Nintendo. The comparison table uses these categories as columns, allowing a user to compare the displayed availability and prices for the same game. The platform labels are treated as the categories represented in the captured interface; the report does not infer additional systems beyond those shown.

5.4 Game Identity and Duplicate Handling

Duplicate prevention is important when catalogue records are entered from more than one source. Matching can use a stable provider identifier where available, or a controlled combination of title, edition and platform attributes. Similar titles, remasters, regional editions and bundles should not be merged automatically without a reliable matching rule.

5.5 Data Normalisation and Validation

Data normalisation reduces inconsistent representations of repeated values such as platform names and genres. Validation should check required fields, valid relationships and expected data types. Records with missing information should be handled explicitly rather than filled with invented values.

5.6 Price and Availability Data

Price and availability information can change by region, currency, edition and date. Each comparison record should retain the source and retrieval timestamp when available. The

user interface should make it clear whether a value is live, recently retrieved, manually curated or sample data. Currency conversion should only be claimed if an implemented and documented conversion source is used.

5.7 Data Quality and Source Governance

The project should maintain a clear distinction between original project data, user-entered information, authorised external data and mock records. Source terms and rate limits must be respected. A process for correcting inaccurate entries and removing duplicates supports the long-term reliability of the database.

5.8 Summary

Cross-platform integration depends on consistent game identity, structured platform associations and transparent source management. These design principles allow the catalogue to grow while reducing duplicate and misleading information.

METHODOLOGY AND ALGORITHMS

6.1 Development Methodology

The project follows an iterative software development approach. The catalogue and discovery functions form the central workflow, while platform associations, comparison views, studio information, deals and wishlist alerts are organised as connected modules. The interface evidence indicates a modular navigation structure and consistent presentation patterns across these functions. Further iterations can extend data coverage, improve source provenance and strengthen automated testing.

6.2 Catalogue Retrieval Workflow

The catalogue retrieval process begins when the frontend requests a collection of game records. The backend validates the request, obtains matching records from its data source and returns a structured response. The frontend renders the response and handles loading, empty and error states.

6.3 Search Algorithm

Purpose: locate catalogue records matching a user's search term.

1. Start.

2. Read the search term entered by the user.

3. Trim unnecessary whitespace and apply the case-handling rule used by the application.

4. Submit the query to the catalogue search function.

5. Compare the query with the supported searchable fields.

6. Collect matching records.

7. Return the result list to the frontend.

8. Display matching games or a no-results message.

9. Stop.

Output: a list of matching game records. The exact searchable fields must match the implementation.

6.4 Filtering Algorithm

Purpose: narrow a catalogue according to selected filter values.

1. Start.

2. Read the selected filter criteria.

3. Retrieve or use the current catalogue result set.

4. For each game, evaluate the implemented filter fields.

5. Retain records satisfying the selected criteria.

6. Return the filtered result set.

7. Update the interface.

8. Stop.

Output: a filtered list of games.

6.5 Game-Platform Association

Purpose: retrieve platform information associated with a game.

1. Receive the selected game identifier.

2. Validate the identifier.

3. Retrieve the game record.

4. Retrieve platform associations linked to that record.

5. Format platform information for display.

6. Return the result.

7. Stop.

Output: game details with the associated platform information.

6.6 Wishlist Management

Purpose: add or remove a game from a user's wishlist.

1. Receive the user identity and game identifier.

2. Validate the request and confirm the game exists.

3. Check whether the wishlist entry already exists.

4. If adding, create the entry only if it does not already exist.

5. If removing, delete or deactivate the matching entry according to the implemented data model.

6. Return the operation status.

7. Refresh the wishlist view.

8. Stop.

Output: updated wishlist state.

6.7 Personal Library Management

Purpose: maintain a user's personal list of games.

1. Receive the user identity and game identifier.

2. Validate the game record.

3. Check whether the game is already in the user's library.

4. Apply the requested add, update or remove operation.

5. Persist the change.

6. Return the updated library information.

7. Stop.

Output: updated personal library. Any status fields, such as owned or completed, should be included only if implemented.

6.8 Spending Tracker Calculation

Purpose: calculate the total spending recorded by a user.

1. Retrieve the user's spending records.

2. Validate that each included amount is numeric and valid.

3. Group records by currency if multiple currencies are supported.

4. Sum the amounts within each currency.

5. Return the total and any period-based breakdown supported by the application.

6. Display the result.

7. Stop.

Output: recorded spending totals. The calculation represents saved entries and is not a bank-account balance.

6.9 Price Comparison Workflow

Purpose: present comparable price records when available.

1. Receive the selected game and relevant platform or store filters.

2. Retrieve available offer records.

3. Confirm that the records refer to the same title and edition where that information is available.

4. Retain currency, region, source and timestamp metadata.

5. Sort or group the records using the application's comparison rule.

6. Display the results with source and freshness information.

7. Stop.

Output: a comparison view. If no verified offer data exists, the application should show an unavailable state rather than fabricate a price.

6.10 Summary

The algorithms describe the main logical operations of the application. They are technology-independent descriptions and must be checked against the corresponding frontend and backend implementation before being presented as exact code behaviour.

TESTING AND RESULTS

7.1 Introduction

Testing and evaluation distinguish visible interface evidence from operations that require direct execution. The supplied screenshots allow inspection of the home dashboard, catalogue, search, platform comparison, studio directory, wishlist alerts and deal listings. They do not provide request logs, database query results for every module, automated test output or performance measurements. The results in this chapter are therefore limited to observable interface behaviour and explicit on-screen persistence statements.

7.2 Testing Methodology

A complete verification process for OmniDB includes frontend functional checks, frontend-to-backend integration tests, relational data validation, duplicate-record checks, price-data freshness checks and browser/device compatibility testing. For the present report, screenshot review is used as the available evidence source. Operations not demonstrated in the supplied screenshots are identified as requiring direct test execution rather than being marked as passed.

7.3 Functional Test Cases

The following table records what can be verified from the supplied running-interface screenshots. A screenshot confirms that a screen and its visible controls were rendered; it does not, by itself, prove every backend transaction, API response or persistence operation.

TEST ID	TEST SCENARIO	EXPECTED RESULT	ACTUAL STATUS
TC-01	Open game catalogue	Catalogue loads without error	Visible in supplied screenshot
TC-02	Search for an existing title	Matching game is displayed	Visible in supplied screenshot
TC-03	Search for an unknown title	No-results state is displayed	Visible in supplied screenshot
TC-04	Open a game details page	Correct game details are displayed	Not demonstrated in supplied screenshots
TC-05	View platform associations	Available platform data is displayed	Visible in supplied screenshot
TC-06	Add a game to wishlist	Wishlist is updated	Visible in supplied screenshot
TC-07	Add a game to personal library	Library is updated	Not demonstrated in supplied screenshots
TC-08	Record a spending entry	Entry is saved and total is updated	Not demonstrated in supplied screenshots
TC-09	Submit invalid or incomplete data	Validation feedback is shown	Not demonstrated in supplied screenshots

TEST ID	TEST SCENARIO	EXPECTED RESULT	ACTUAL STATUS
TC-10	Backend/data source unavailable	Application handles failure gracefully	Not demonstrated in supplied screenshots

7.4 Integration Testing

Integration testing should verify that frontend actions invoke the intended backend operations, that the backend validates identifiers and user-specific changes, and that PostgreSQL records are created, updated or removed correctly. For the wishlist, the supplied screen explicitly identifies persistent alerts in the user_watchlist table. A complete integration record would additionally include request/response examples and database observations for add, edit, delete and target-price-trigger operations.

7.5 Results and Discussion

The supplied screenshots provide visual evidence of the home dashboard, universal search, catalogue filtering, platform comparison, studio profiles, wishlist target-price alerts and deal listings. The dashboard snapshot displays six platform/store categories, 1,075 store listings, 1,012 deals and 545 historical record lows. The wishlist page explicitly states that persistent alerts are stored in the PostgreSQL user_watchlist table. These are interface-observed values at the time of capture and should not be interpreted as independently audited live-market totals. No API traces, benchmark measurements or formal user-study results were supplied; accordingly, this chapter reports observable interface outcomes and identifies deeper integration and performance tests as work requiring separate execution.

7.6 Limitations of Testing

A complete evaluation requires repeatable tests against the configured application and database, with dates, environment details and observed results recorded. The current evidence set does not include load testing, security testing, formal usability evaluation, API traces or proof of real-time price refresh. These are evaluation limitations, not evidence that the visible application functions are absent.

7.7 Summary

Testing establishes whether the catalogue, search, details, platform information and personal features work together. The final submission should include actual results and evidence rather than treating the proposed test plan as completed test execution.

CONCLUSION AND FUTURE SCOPE

8.1 Conclusion

OmniDB – Universal Game Database is a cross-platform game information and discovery application built around the vision “One game. Every platform. One database.” It consolidates game discovery, platform associations, store listings, developer and publisher information, deals and user-defined target-price alerts in one interface. The screenshots demonstrate a multi-store catalogue, search by title and metadata, platform-wise price comparison, studio profiles, historical-low deal indicators and a PostgreSQL-backed wishlist alert reference. The project team reports completion of frontend and backend development. The report focuses on functions visible in the supplied running-system evidence and treats unverified integrations, performance measurements and additional personal-management features as areas for further validation.

The value of the application depends on consistent game records, accurate platform associations, reliable backend operations and transparent data sources. The report therefore distinguishes the project's intended capabilities from details that must be verified against the final implementation, especially external integrations and live price data.

8.2 Limitations

The completeness of the catalogue depends on the data maintained by the project. Price and availability information may change and can differ by region, currency and edition. Records from different sources may contain inconsistent names or metadata. The exact backend technology, schema, source integrations and deployment configuration should be documented from the final implementation. Spending totals depend on the accuracy of the entries recorded by the user.

8.3 Future Scope

Possible future enhancements include expanding the catalogue, integrating additional authorised data providers, improving duplicate detection, recording source timestamps, adding region-aware price comparison, supporting more detailed library statuses, providing spending summaries by month or platform, adding export options, improving accessibility, and strengthening authentication and security. These are future possibilities and should not be described as completed features unless implemented.

8.4 Final Remarks

A unified game database can make cross-platform game information easier to explore and organise. OmniDB provides a foundation for bringing catalogue discovery and personal game management into one application. Continued improvements in data quality, source governance and usability can strengthen its usefulness as the catalogue grows.

REFERENCES

The references below identify the project evidence and core database documentation relevant to the implementation described in this report. The project screenshots are treated as primary evidence for the visible interface. No third-party storefront API is claimed unless separately documented by the project team.

REFERENCES

[1] A. D'Angelo, C. Di Sipio, C. Politowski, and R. Rubei, "PlayMyData: A Curated Dataset of Multi-Platform Video Games," *Proceedings of the 21st IEEE/ACM International Conference on Mining Software Repositories (MSR)*, pp. 525–529, 2024. doi: 10.1145/3643991.3644869.

[2] J. H. Lee, R. I. Clarke, and A. Perti, "Empirical Evaluation of Metadata for Video Games and Interactive Media," *Journal of the Association for Information Science and Technology*, vol. 66, no. 12, pp. 2609–2625, 2015. doi: 10.1002/asi.23357.

[3] J. H. Lee, R. I. Clarke, and S. Rossi, "A Qualitative Investigation of Users' Discovery, Access, and Organization of Video Games as Information Objects," *Journal of Information Science*, vol. 42, no. 6, pp. 833–850, 2016. doi: 10.1177/0165551515618594.

[4] J. Ryan, E. Kaltman, T. Hong, K. Isbister, M. Mateas, and N. Wardrip-Fruin, "GameNet and GameSage: Videogame Discovery as Design Insight," *Proceedings of the DiGRA/FDG 2016 Conference*, 2016. doi: 10.26503/dl.v2016i1.760.

[5] E. Kaltman, N. Wardrip-Fruin, M. Mastroni, H. Lowood, G. De Groat, G. Edwards, M. Barrett, and C. Caldwell, "Implementing Controlled Vocabularies for Computer Game Platforms and Media Formats in SKOS," *Cataloging & Classification Quarterly*, 2016. doi: 10.1080/19386389.2016.1167494.

[6] R. T. Fielding, "Architectural Styles and the Design of Network-Based Software Architectures," Ph.D. dissertation, University of California, Irvine, 2000. Available: https://ics.uci.edu/~fielding/pubs/dissertation/top.htm

[7] Valve Corporation, "Steamworks Web API Reference and Documentation." Available: https://partner.steamgames.com/doc/webapi

[8] International Game Developers Association (IGDB), "IGDB API Documentation." Available: https://api-docs.igdb.com/

[9] T. Bray, Ed., “The JavaScript Object Notation (JSON) Data Interchange Format,” RFC 8259, Internet Engineering Task Force (IETF), Dec. 2017. Available: https://www.rfc-editor.org/rfc/rfc8259

[10] R. Elmasri and S. B. Navathe, *Fundamentals of Database Systems*, 7th ed. Pearson, 2016.

The following implementation figures are original screenshots supplied by the project team from the running OmniDB interface. They are placed in the relevant implementation sections and are used as visual evidence of the displayed features. The screenshots show the application interface and visible status information; they are not treated as substitutes for source-code review or formal backend testing.

APPENDIX A – SYSTEM DESIGN DIAGRAMS

The following diagrams present the logical organisation and principal interactions of OmniDB. They are conceptual design views prepared for the report; exact API names and every physical database table are not implied unless explicitly identified in the running interface.

Figure 3.1 – High-Level System Architecture

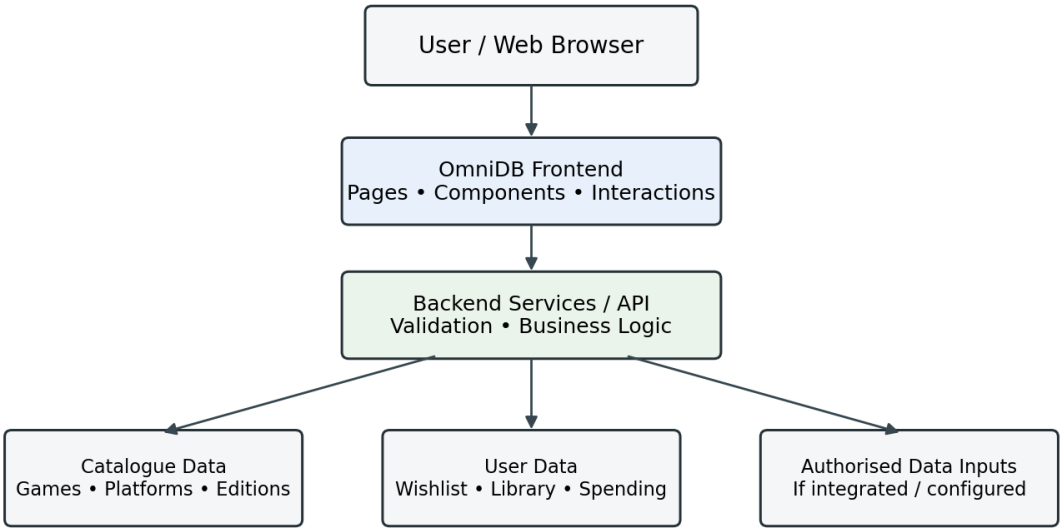


Figure 3.1: High-Level System Architecture

Figure 3.2 – Context-Level Data Flow

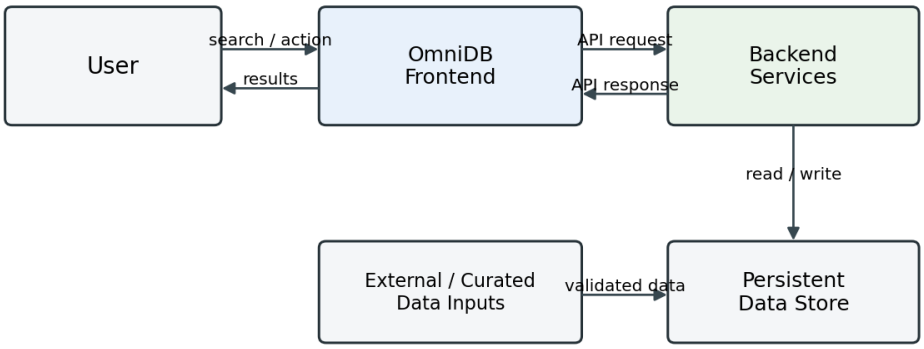


Figure 3.2: Context-Level Data Flow

Figure 3.6 – User Use Cases

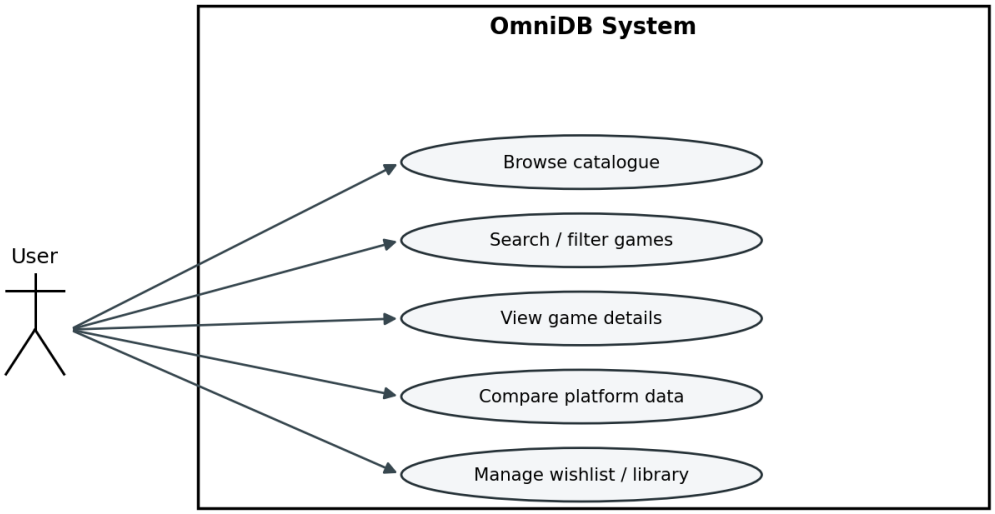


Figure 3.6: User Use Cases

Figure 3.3 – Game Discovery Activity Flow

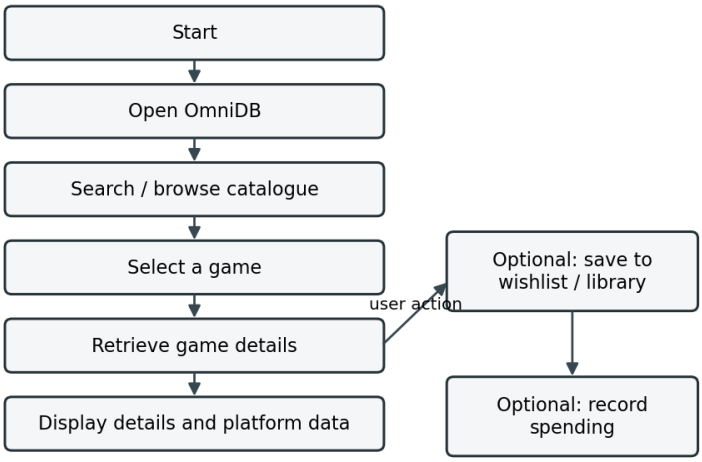


Figure 3.3: Game Discovery Activity Flow

Figure 3.4 – Sequence Diagram for Catalogue Search

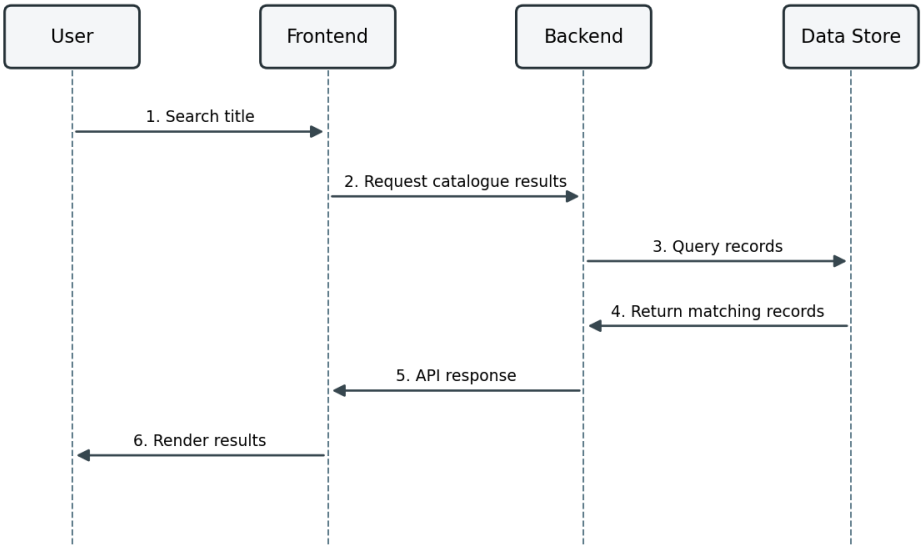


Figure 3.4: Sequence Diagram for Catalogue Search

Figure 3.5 – Conceptual Entity Relationship Model

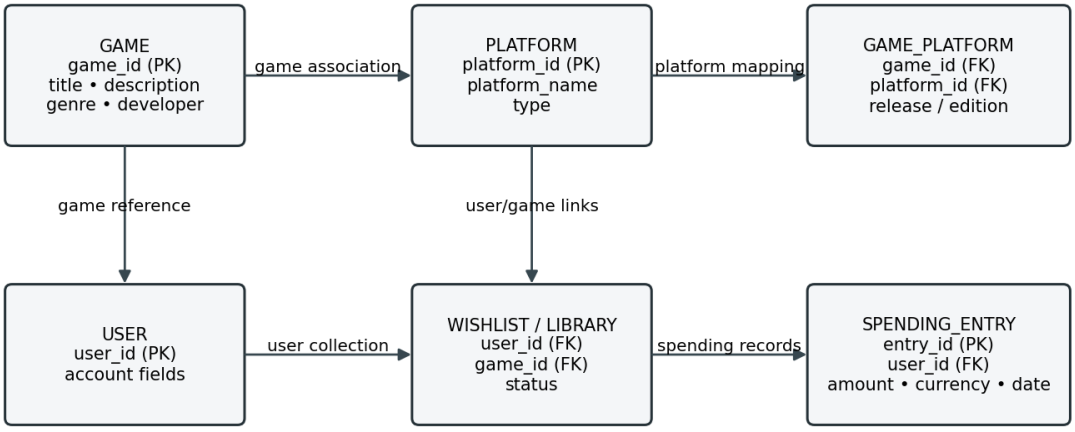


Figure 3.5: Conceptual Entity Relationship Model

APPENDIX B – DATA QUALITY AND VALIDATION PLAN

A cross-platform catalogue must preserve meaningful differences between games, editions, platform releases and storefront listings. The following validation rules provide a practical checklist for maintaining catalogue quality.

AREA	VALIDATION RULE	REASON
Game identity	Use a stable internal game identifier; do not rely only on display title.	Prevents accidental duplicate or merged records.
Edition identity	Keep standard, deluxe, remake, remaster and bundle distinctions where applicable.	Preserves meaningful release differences.
Platform mapping	Associate a game/release with explicit supported platform records.	Avoids unsupported platform assumptions.
Metadata	Validate required fields and data types before saving.	Reduces incomplete and malformed records.
Source provenance	Store source name and retrieval time for imported values where available.	Supports traceability and freshness checks.
Price records	Keep amount, currency, region, edition, source and timestamp together.	Makes comparisons meaningful.
Duplicate candidates	Flag uncertain matches for review instead of auto-merging.	Reduces false merges.
User records	Keep wishlist, library and spending records associated with the correct user.	Protects personal data.

APPENDIX C – SECURITY, PRIVACY AND RELIABILITY CONSIDERATIONS

C.1 User Data Separation

Personal library, wishlist and spending information should be separated from the public game catalogue. Backend operations should verify that a user can access only records permitted for that account.

C.2 Input Validation

Search terms, identifiers, amounts and other user-supplied values should be validated on the server as well as the client. Invalid requests should return understandable errors.

C.3 Secret Management

API keys, database credentials and other secrets should not be placed in public frontend source code or screenshots. Environment-specific configuration should be used where appropriate.

C.4 External Data Reliability

External providers may change their responses, impose rate limits or become unavailable. The application should handle missing or stale information without displaying fabricated values.

C.5 Backup and Recovery

If persistent storage is used, the project should define an appropriate backup and recovery approach and test restoration procedures before production use.

C.6 Privacy and Data Minimisation

Only collect personal information needed for the application's functions. Spending records should be treated as user data and protected accordingly.

APPENDIX D – SUSTAINABLE DEVELOPMENT GOAL RELEVANCE

OmniDB is primarily a software information-management project. Its contribution to the Sustainable Development Goals should be described in relation to the project's educational and technological learning outcomes, not as a claim of direct measurable social impact.

SDG	RELEVANCE TO THE PROJECT	PROJECT CONNECTION
SDG 9 – Industry, Innovation and Infrastructure	Primary alignment	The project applies software engineering, data organization and cross-platform information integration to build a digital information system.
SDG 4 – Quality Education	Supporting alignment	The project provides practical learning in requirements analysis, frontend/backend development, data modelling, testing and documentation.
SDG 8 – Decent Work and Economic Growth	Indirect alignment	The project develops transferable software development and problem-solving skills relevant to digital technology work.

APPENDIX E – PROJECT EVALUATION INDICATORS

OUTCOME	POSSIBLE INDICATOR	EVIDENCE
Catalogue usability	Time required to locate a selected game	Observed task test
Search quality	Relevant results returned for representative search terms	Test queries and result review
Data consistency	Number of duplicate or incomplete records identified	Catalogue audit

OUTCOME	POSSIBLE INDICATOR	EVIDENCE
Platform information	Correctness of platform associations in sampled records	Comparison against authorised source records
Personal feature reliability	Successful save, update and retrieval of user records	Frontend/backend integration tests
Spending calculation	Correct aggregation of entered amounts by currency	Known-input calculation test
Backend reliability	Correct handling of valid and invalid requests	API test evidence